

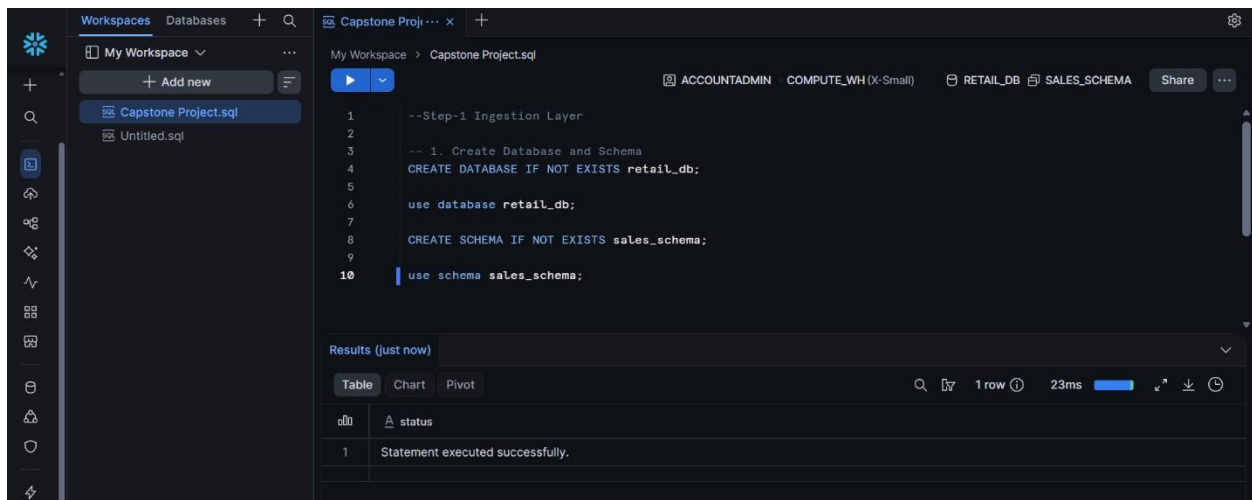
SNOWFLAKE

CAPSTONE PROJECT

A. Ingestion Layer

First, I created the foundational objects in Snowflake: a database, a schema, a staging table for the raw CSV data and an internal stage.

1. Create Database and Schema



The screenshot shows the Snowflake SQL Editor interface. The left sidebar displays the workspace with 'Capstone Project.sql' selected. The main editor area contains the following SQL code:

```
--Step-1 Ingestion Layer
-- 1. Create Database and Schema
CREATE DATABASE IF NOT EXISTS retail_db;

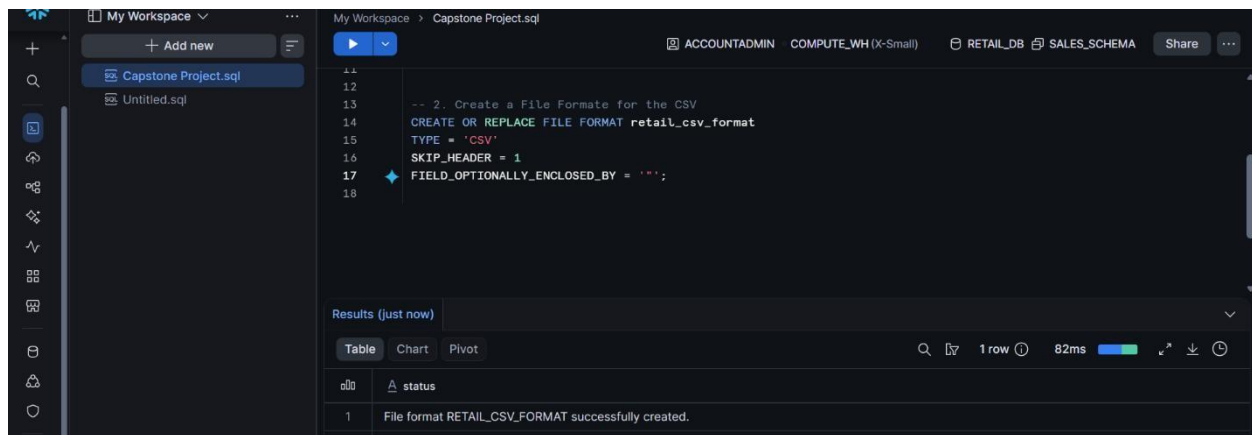
use database retail_db;

CREATE SCHEMA IF NOT EXISTS sales_schema;

use schema sales_schema;
```

The 'Results (just now)' section shows a single row with the status 'Statement executed successfully.' The execution time is 23ms.

2. Create a File Format for the CSV



The screenshot shows the Snowflake SQL Editor interface. The left sidebar displays the workspace with 'Capstone Project.sql' selected. The main editor area contains the following SQL code:

```
-- 2. Create a File Format for the CSV
CREATE OR REPLACE FILE FORMAT retail_csv_format
TYPE = 'CSV'
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '';
```

The 'Results (just now)' section shows a single row with the status 'File format RETAIL_CSV_FORMAT successfully created.' The execution time is 82ms.

3. Create an Internal Stage

The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
19
20 -- 3. Create an Internal Stage
21 create or replace stage retail_stage
22 file_format = retail_csv_format;
23
```

The 'Results (just now)' panel shows a single row with the status:

status
1 Stage area RETAIL_STAGE successfully created.

4. Create Raw Table

The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
23
24 -- 4. Create Raw Table
25 CREATE OR REPLACE TABLE stg_orders(
26 Row_ID NUMBER(3),
27 Order_ID VARCHAR(50),
28 Order_Date DATE,
29 Ship_Date DATE,
30 Ship_Mode VARCHAR(200),
31 Customer_ID VARCHAR(50),
32 Customer_Name VARCHAR(300),
33 Segment VARCHAR(100),
34 Country VARCHAR(100),
35 City VARCHAR(100),
36 State VARCHAR(100),
37 Postal_Code NUMBER(10),
38 Region VARCHAR(100),
39 Product_ID VARCHAR(100),
40 Category VARCHAR(100),
41 Sub_Category VARCHAR(100),
42 Product_Name VARCHAR(100),
43 Price NUMBER(10,4),
44 Quantity NUMBER(3).
```

The 'Results (just now)' panel shows a single row with the status:

status
1 Table STG_ORDERS successfully created.

5. Load Data and Verify Loaded Data

The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
47
48 -- 5. Load Data
49 copy into stg_orders
50 from @retail_stage
51 file_format = (format_name='retail_csv_format');
52
```

The 'Results (just now)' panel shows a single row with the status:

status
1 Data loaded successfully.

The screenshot also shows a second workspace named 'Capstone Project.sql' with the following code:

```
53
54 -- 6. Verify Loaded Data
55 SELECT Count(*)
56 From STG_ORDERS;
57
```

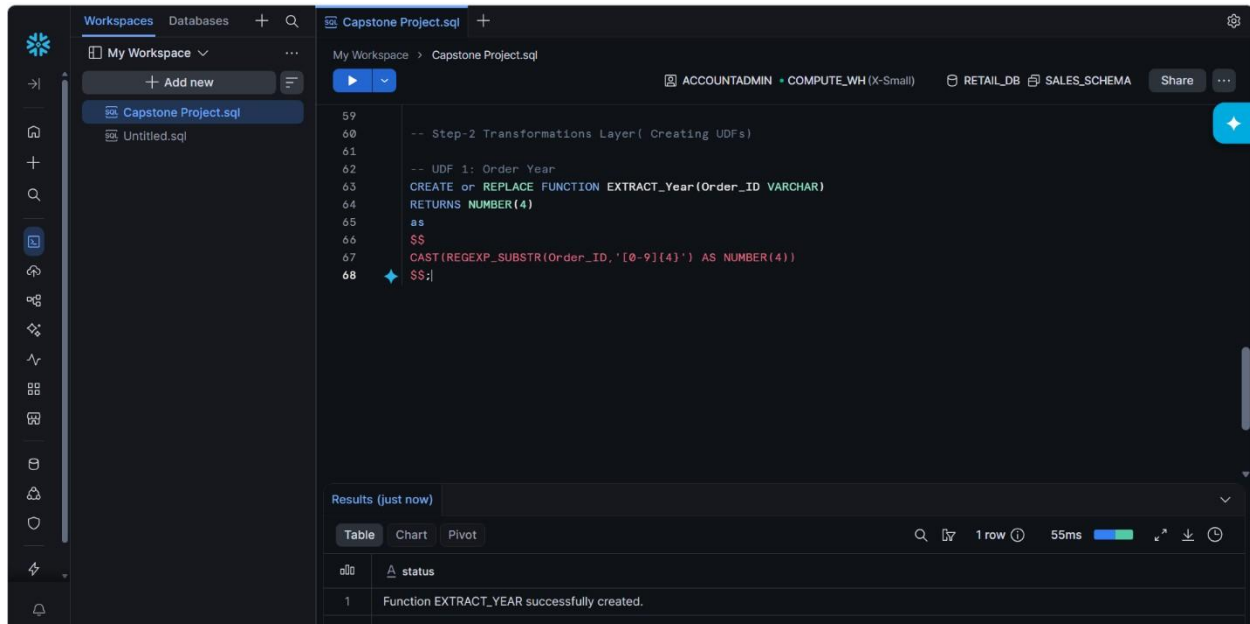
The 'Results (just now)' panel shows a single row with the count:

Count(*)
1 1

B. Transformations Layer (Creating UDFs)

1. UDF 1: Order Year

The first UDF dynamically extracts the 4-digit year directly from the ORDER_ID string (Eg: isolating “2016” from “CA-2016-123”).



The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

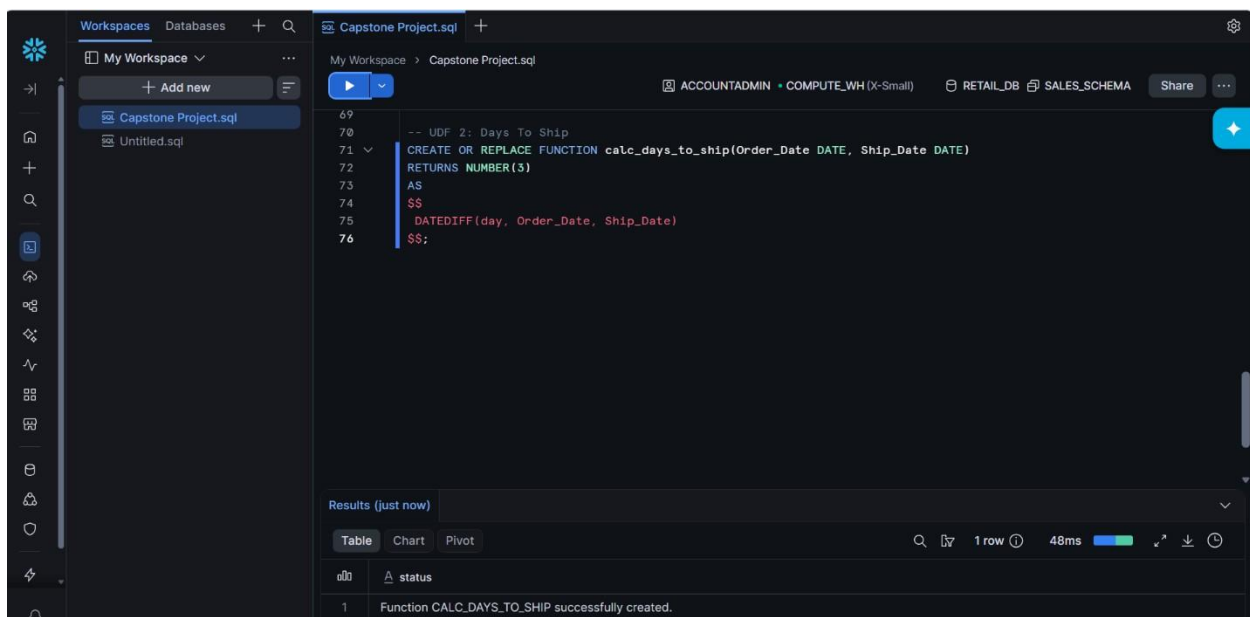
```
59 -- Step-2 Transformations Layer( Creating UDFs)
60
61 -- UDF 1: Order Year
62
63 CREATE or REPLACE FUNCTION EXTRACT_Year(Order_ID VARCHAR)
64 RETURNS NUMBER(4)
65 AS
66 $$
67 CAST(REGEXP_SUBSTR(Order_ID, '[0-9]{4}') AS NUMBER(4))
68 $$;
```

The results pane shows a single row with the status: 'Function EXTRACT_YEAR successfully created.'

status
Function EXTRACT_YEAR successfully created.

2. UDF 2: Days to Ship

The second UDF calculates the logistics turn around time by finding the exact number of days between the order placement and shipping dates.



The screenshot shows the same SQL IDE interface with the workspace 'Capstone Project.sql'. The SQL editor contains the following code:

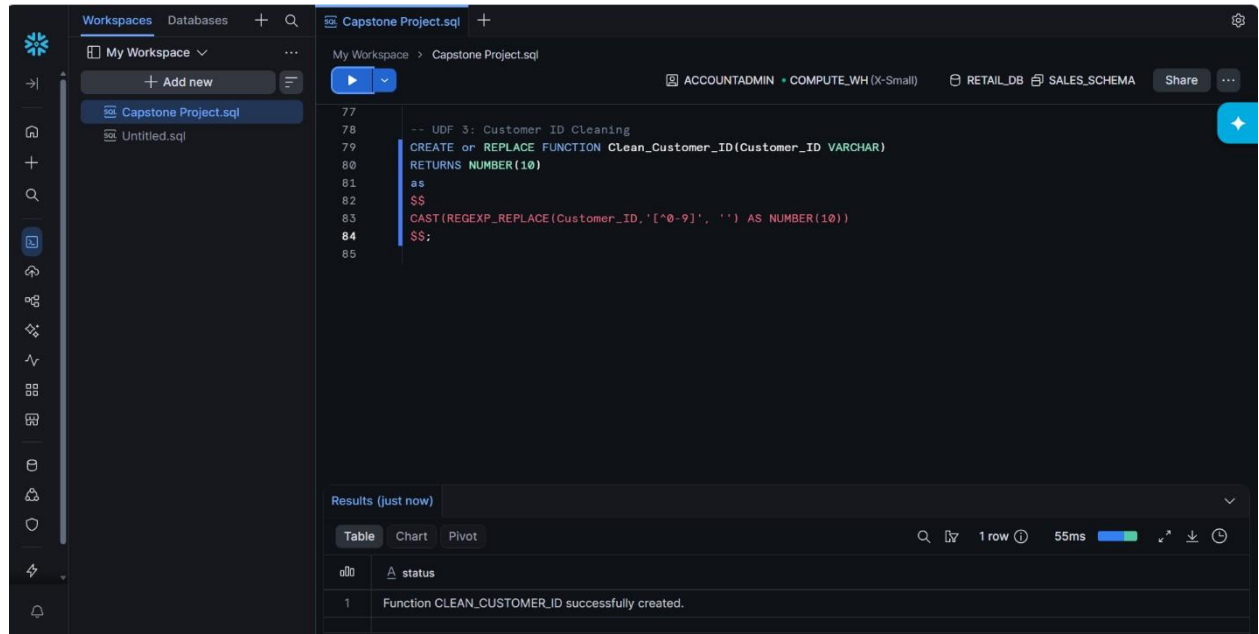
```
69 -- UDF 2: Days To Ship
70
71 CREATE OR REPLACE FUNCTION calc_days_to_ship(Order_Date DATE, Ship_Date DATE)
72 RETURNS NUMBER(3)
73 AS
74 $$
75 DATEDIFF(day, Order_Date, Ship_Date)
76 $$;
```

The results pane shows a single row with the status: 'Function CALC_DAYS_TO_SHIP successfully created.'

status
Function CALC_DAYS_TO_SHIP successfully created.

3. UDF 3: Customer ID Cleaning

The CLEAN_ID UDF uses regular expressions to strip out all alphabetical prefixes and suffixes from strings, ensuring cleaner joins by converting IDs to integers.



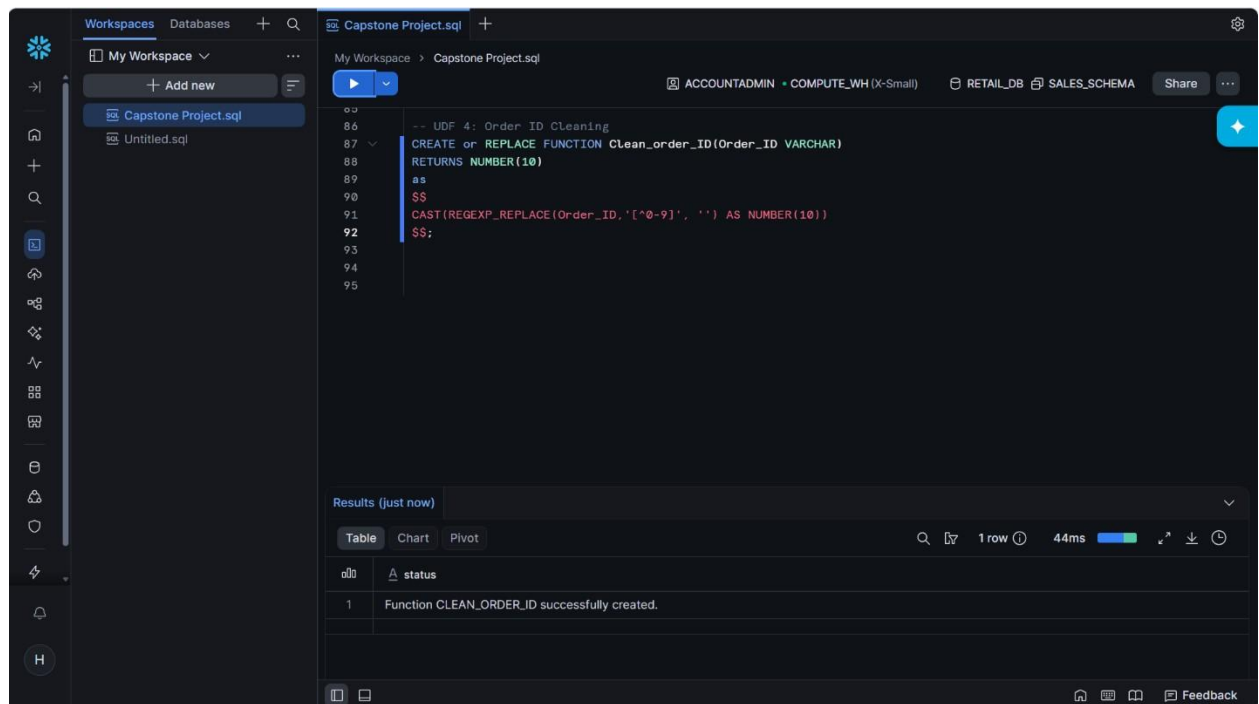
The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
-- UDF 3: Customer ID Cleaning
CREATE or REPLACE FUNCTION Clean_Customer_ID(Customer_ID VARCHAR)
RETURNS NUMBER(10)
as
$$
CAST(REGEXP_REPLACE(Customer_ID, '[^0-9]', '' ) AS NUMBER(10))
$$;
```

The results pane shows a single row with the status: 'Function CLEAN_CUSTOMER_ID successfully created.'

status
Function CLEAN_CUSTOMER_ID successfully created.

4. UDF 4: Order ID Cleaning



The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
-- UDF 4: Order ID Cleaning
CREATE or REPLACE FUNCTION Clean_order_ID(Order_ID VARCHAR)
RETURNS NUMBER(10)
as
$$
CAST(REGEXP_REPLACE(Order_ID, '[^0-9]', '' ) AS NUMBER(10))
$$;
```

The results pane shows a single row with the status: 'Function CLEAN_ORDER_ID successfully created.'

status
Function CLEAN_ORDER_ID successfully created.

5. UDF 5: Product ID Cleaning

The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

```
-- UDF 5: Product ID Cleaning
CREATE or REPLACE FUNCTION Clean_Product_ID(Product_ID VARCHAR)
RETURNS NUMBER(10)
AS
$$
CAST(REGEXP_REPLACE(Product_ID, '[^0-9]', '' ) AS NUMBER(10))
$$;
```

The results pane shows a single row with the status: 'Function CLEAN_PRODUCT_ID successfully created.'

status
Function CLEAN_PRODUCT_ID successfully created.

6. UDF 6: Sales Calculation

The CALC_SALES UDF provides a consistent, centralized mathematical formula to drive total revenue per line item by multiplying price by quantity.

The screenshot shows a SQL IDE interface with a workspace named 'Capstone Project.sql'. The SQL editor contains the following code:

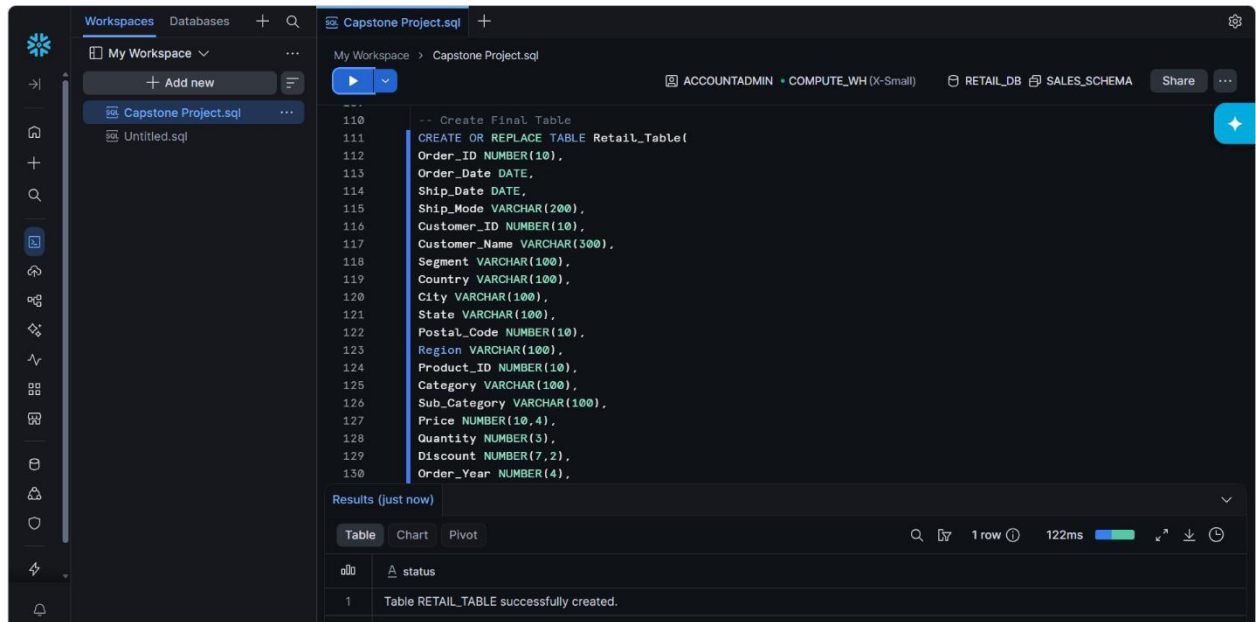
```
-- UDF 6: Sales Calculation
CREATE OR REPLACE FUNCTION calc_sales(Price NUMBER(10,4), Quantity NUMBER(3))
RETURNS NUMBER(15,5)
AS
$$
Price*Quantity
$$;
```

The results pane shows a single row with the status: 'Function CALC_SALES successfully created.'

status
Function CALC_SALES successfully created.

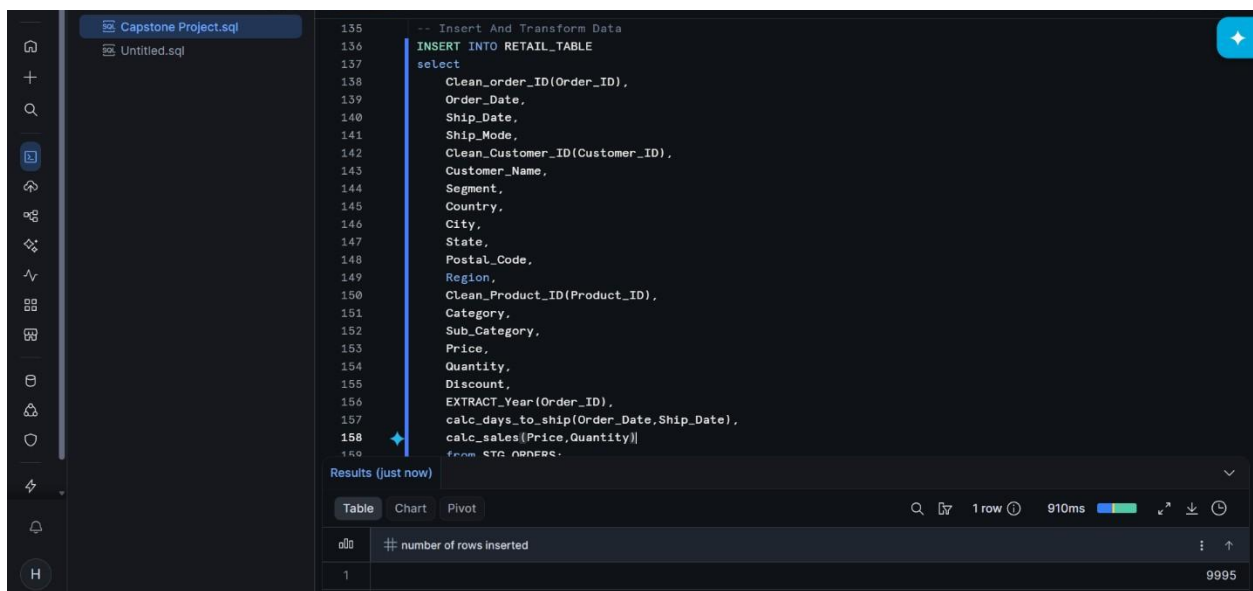
7. Create Final Table

This defines the final, production ready schema for the analytics layer according to the strict business requirements and standardized data types.



8. Insert And Transform Data

This query executes the initial full data load, migrating records from the staging table to the final table.



C. Processing Layer

1. Create Stream and Scheduled Task with MERGE Statement

The Stream continuously tracks any new inserts, updates, or deletes occurring in the raw staging table to enable Change Data Capture (CDC). The scheduled task automatically merges only these newly detected delta changes into the final target table, ensuring dashboards remain up to date with minimal compute overhead.

```

164
165 -- C. Processing Layer
166
167 -- 1. Stream
168 CREATE OR REPLACE STREAM Retail_Stream
169 ON TABLE STG_ORDERS;
170
171 -- 2. Create The Scheduled Task With MERGE Statement
172 CREATE OR REPLACE TASK Daily_Sales_Merge_Task
173 WAREHOUSE = COMPUTE_WH
174 SCHEDULE = '1440 MINUTE'
175 WHEN
176     SYSTEM$STREAM_HAS_DATA('Orders_Stream')
177 AS MERGE INTO RETAIL_TABLE tgt
178 using (
179     SELECT
180         Clean_order_ID(Order_ID) as Order_id,
181         Order_Date, Ship_Date, Ship_Mode, Clean_Customer_ID(Customer_ID) as Customer_ID,
182         Customer_Name, Segment, Country, City, State, Postal_Code, Region,
183         Clean_Product_ID(Product_ID) as Product_ID, Category,
184         Sub_Category, Price, Quantity, Discount, EXTRACT_Year(Order_ID) as Order_Year,
185         calc_days_to_ship(Order_Date, Ship_Date) as Days_To_Ship,
186         calc_sales(Price, Quantity) as Sales,
187         METADATA$ACTION,
188         METADATA$ISUPDATE
189     FROM ORDERS_STREAM
190 ) src
191 ON tgt.order_id = src.order_id AND tgt.product_id = src.product_id
192 -- Handle Updates
193 WHEN MATCHED AND src.METADATA$ACTION = 'INSERT' AND src.METADATA$ISUPDATE = TRUE THEN
194     UPDATE SET
195         tgt.Ship_Date = src.Ship_Date,
196         tgt.Ship_Mode = src.Ship_Mode,
197         tgt.Price = src.Price,
198         tgt.Quantity = src.Quantity,
199         tgt.Discount = src.Discount,
200         tgt.Days_To_Ship = src.Days_To_Ship,
201         tgt.Sales = src.Sales
202 -- Handle Deletes
203 WHEN MATCHED AND src.METADATA$ACTION = 'DELETE' THEN
204     DELETE
205 -- Handle Inserts
206 WHEN NOT MATCHED AND src.METADATA$ACTION = 'INSERT' THEN
207     INSERT(
208         Order_id, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Customer_Name,
209         Segment, Country, City, State, Postal_Code, Region, Product_ID, Category,
210         Sub_Category, Price, Quantity, Discount, Order_Year, Days_To_Ship, Sales)
211     VALUES(
212         src.Order_id, src.Order_Date, src.Ship_Date, src.Ship_Mode, src.Customer_ID, src.Customer_Name,
213         src.Segment, src.Country, src.City, src.State, src.Postal_Code, src.Region, src.Product_ID, src.Category,
214         src.Sub_Category, src.Price, src.Quantity, src.Discount, src.Order_Year, src.Days_To_Ship, src.Sales);
215
216
217
218
219
220
221
222

```

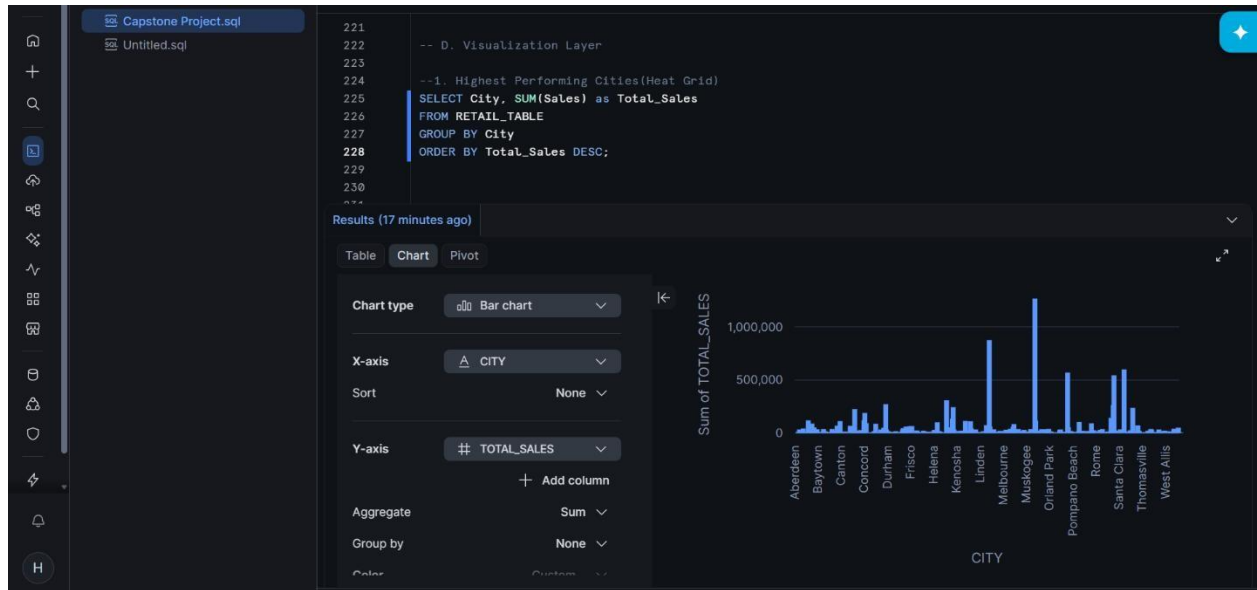
Results (3 minutes ago)

Table	Chart	Pivot
1	status	Task DAILY_SALES_MERGE_TASK successfully created.

D. Visualization Layer

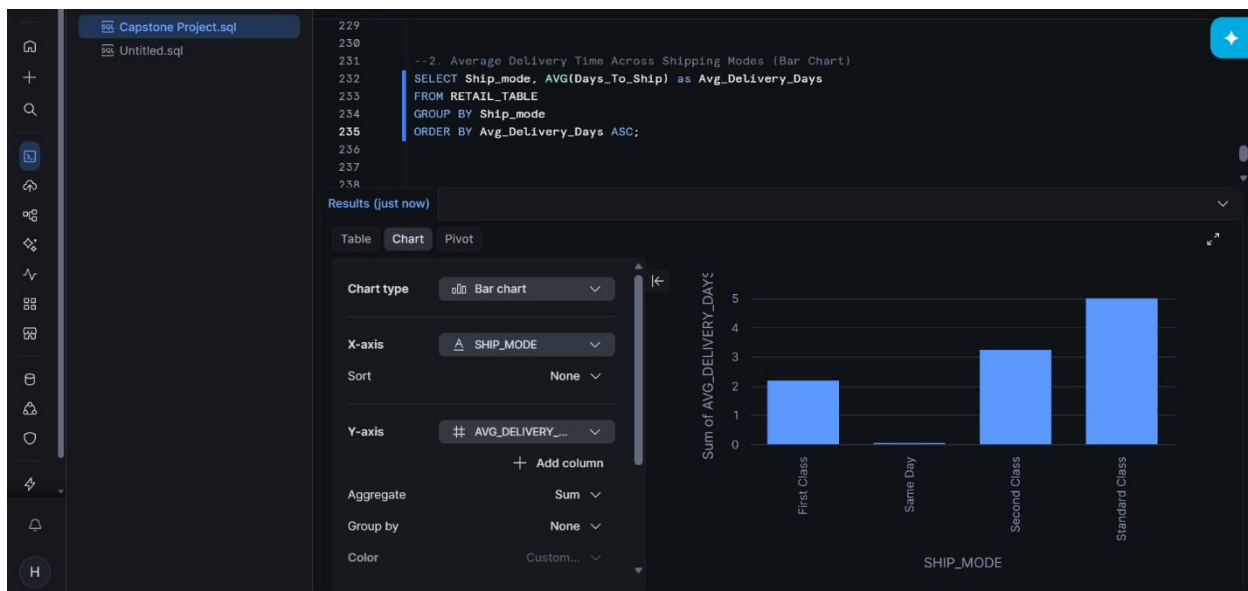
1. Highest Performing Cities (Heat Grid)

The first query aggregates the derived sales metric to identify top geographic revenue hubs for targeted marketing gird mappings.

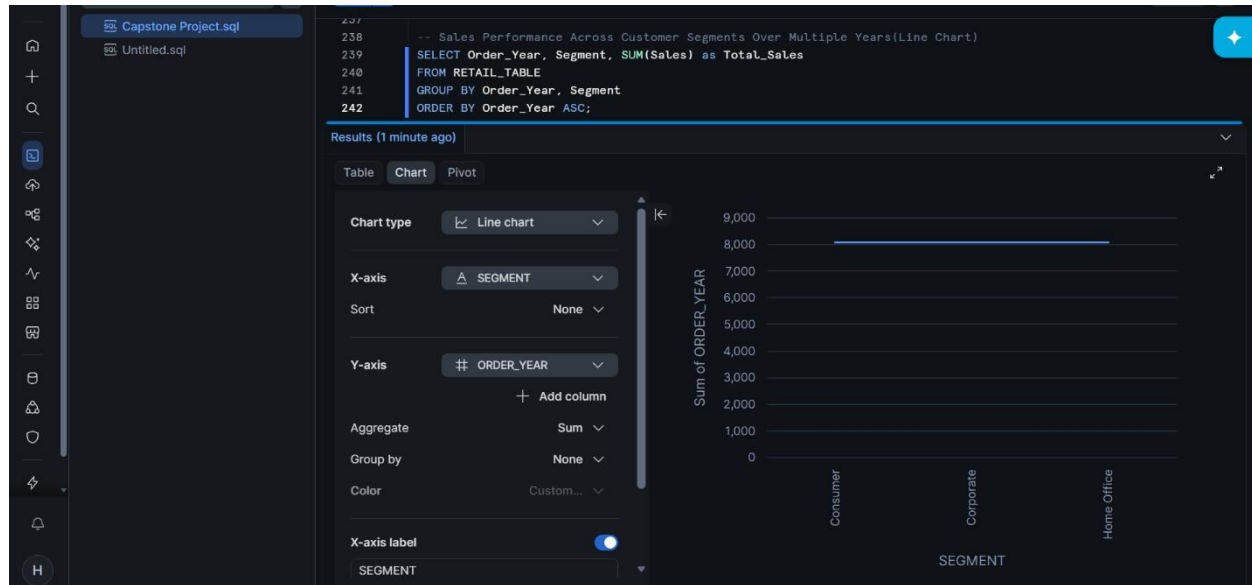


2. Average Delivery Time Across Shipping Modes (Bar Chart)

The second query calculates efficiency across logistics channels by averaging the previously computed DAYS_TO_SHIP metric.

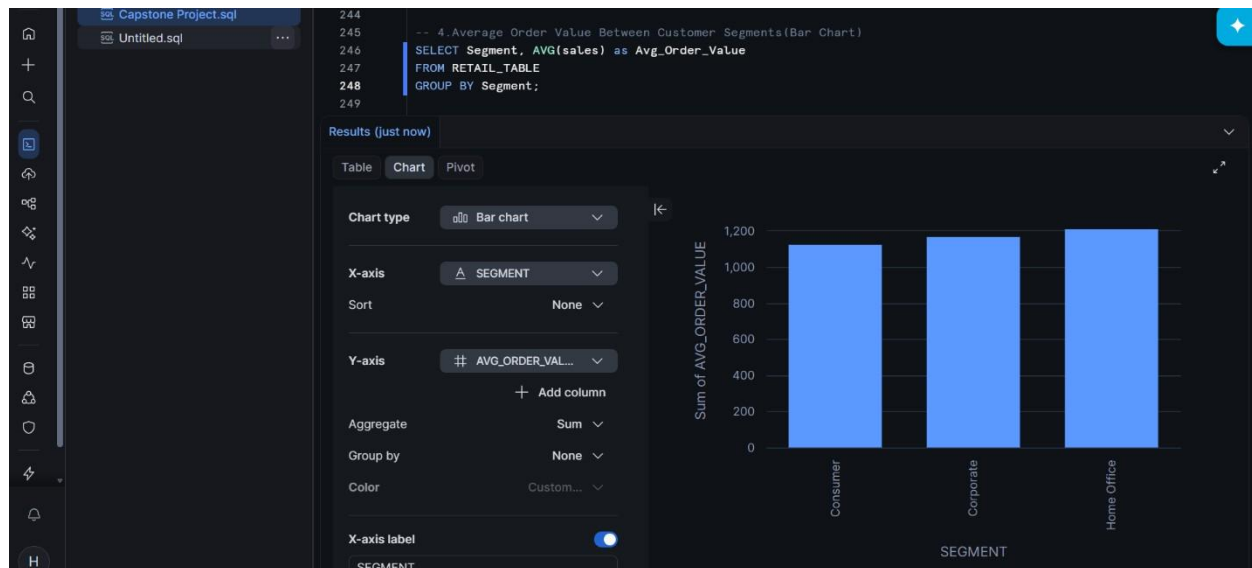


3. Sales Performance Across Customer Segments Over Multiple Years (Line Chart) The third query utilizes the derived ORDER_YEAR UDF column the plot historical growth trends over time on a line chart.



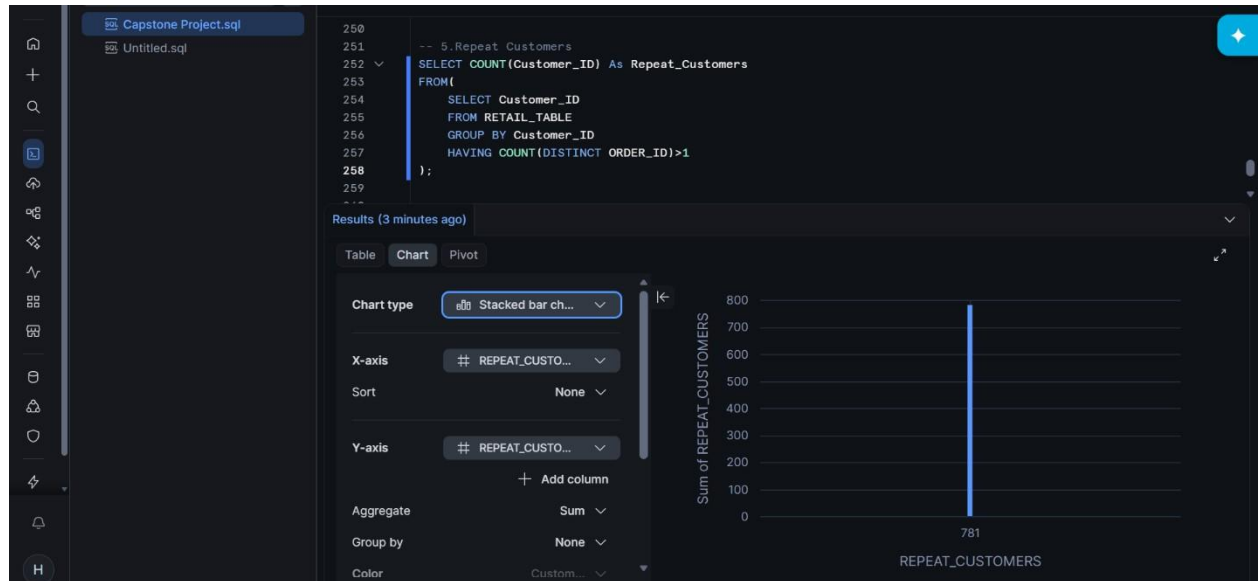
4. Average Order Value Between Customer Segments (Bar Chart)

The fourth query determines the purchasing power of different audience demographics to inform tailored pricing and retention strategies on a bar chart.



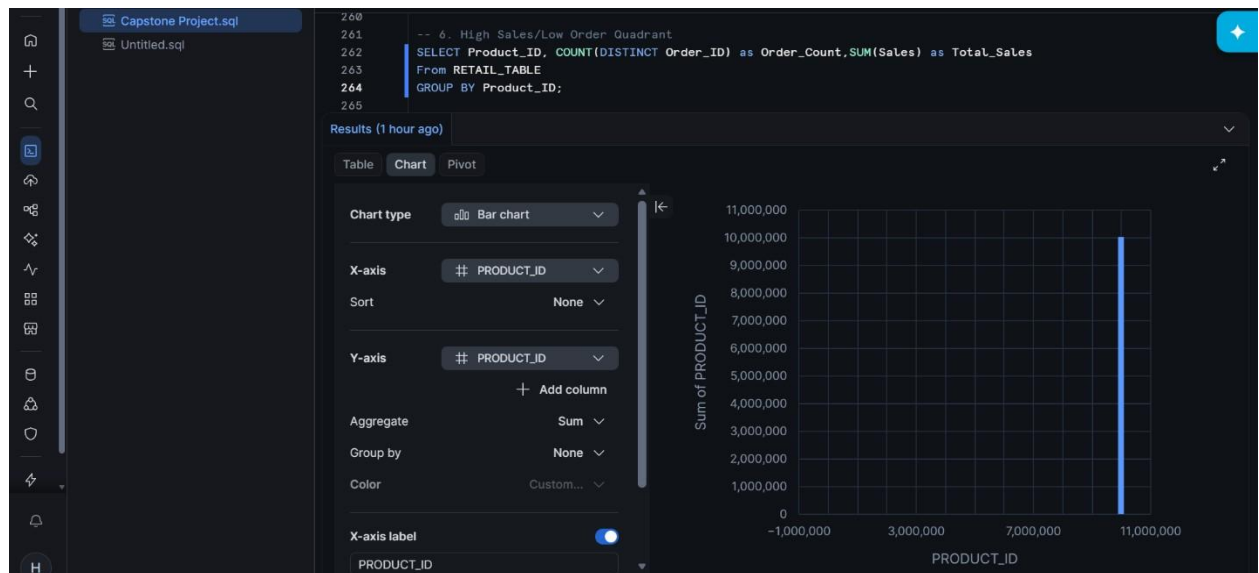
5. Repeat Customers

The fifth query calculates retention effectiveness identifying individuals who have completed more than one distinct transaction.



6. High Sales/Low Order Quadrant

The sixth query structures data for a scatter plot quadrant analysis to uncover niche, high luxury products that generate massive revenue from very few purchases.



E. AI/ML Layer

NOTE: Due to Snowflake Trial account AI function, Stream lit, ML Forecasting, are not available.

The AI/ML layer leverages Snowflake Cortex capabilities including Cortex Analyst, Cortex LLM, ML Forecasting, and Cortex Search to enable intelligent, conversational analytics.

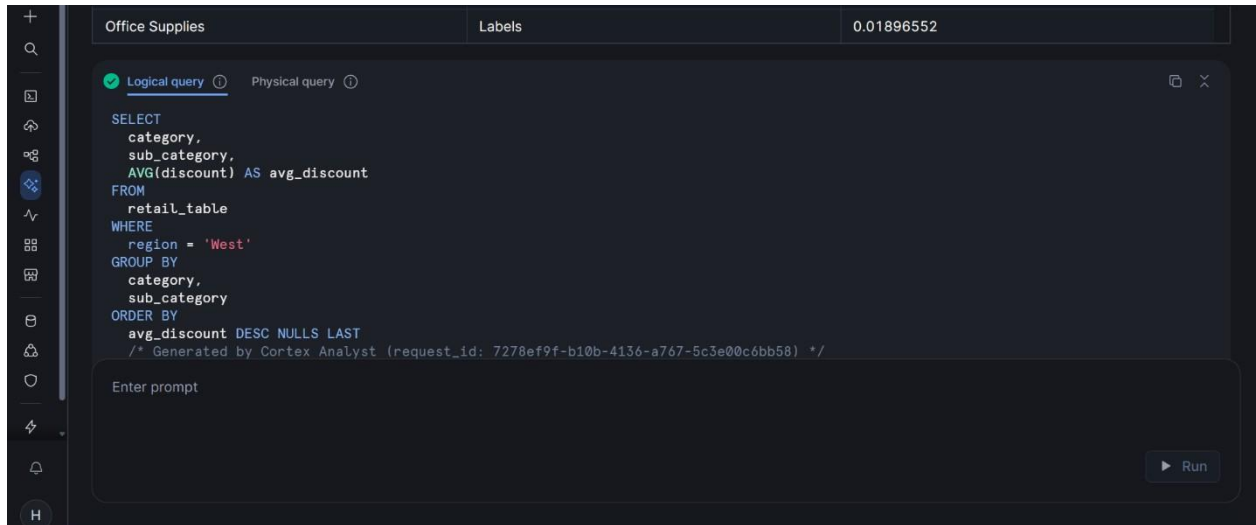
1. Retail Sales Cortex Analyst

The screenshot displays the Snowflake Cortex Analyst interface. At the top, the title bar reads "Cortex Analyst RETAIL_SALES_VIEW". Below this, the "Suggestions" tab is active, showing a "Playground" section. The "User" input field contains the question: "What was total revenue by region last year based on order_date?". The "Cortex Analyst" response provides an interpretation of the question and a SQL query. The query is as follows:

```
SELECT
  region,
  SUM(sales) AS total_revenue
FROM
  retail_table
WHERE
  order_date >= DATE_TRUNC('YEAR', DATEADD(YEAR, -1, CURRENT_DATE))
  AND order_date < DATE_TRUNC('YEAR', CURRENT_DATE)
GROUP BY
  region
ORDER BY
  total_revenue DESC NULLS LAST
/* Generated by Cortex Analyst (request_id: 8c1da555-340b-4854-959d-f79b2804259e) */
```

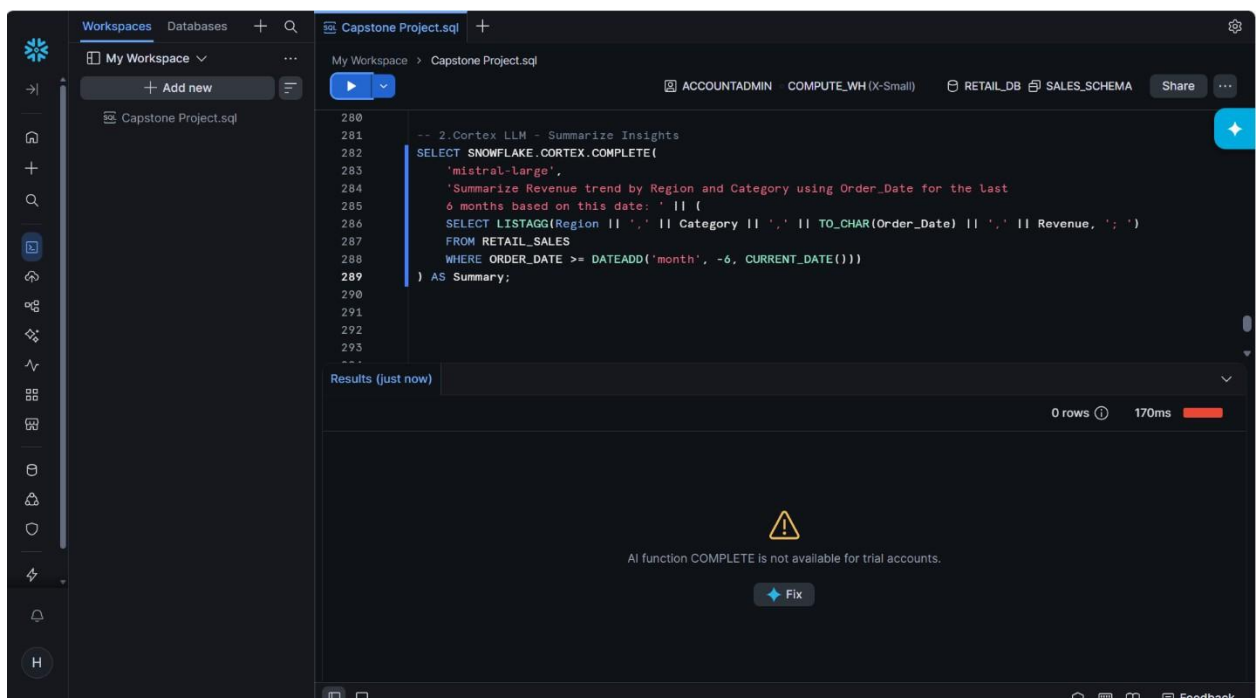
Below the query, there is a "Logical query" tab and a "Physical query" tab. The "Logical query" tab is selected, showing the query. Below the query, there is a "Run" button. The "User" input field contains the question: "Which Category and sub_category combinations have the highest average discount in the west region?". The "Cortex Analyst" response provides an interpretation of the question and a table of results.

CATEGORY	SUB_CATEGORY	AVG_DISCOUNT
Technology	Machines	0.30256410
Office Supplies	Binders	0.28174098
Furniture	Bookcases	0.22875000
Furniture	Tables	0.20000000



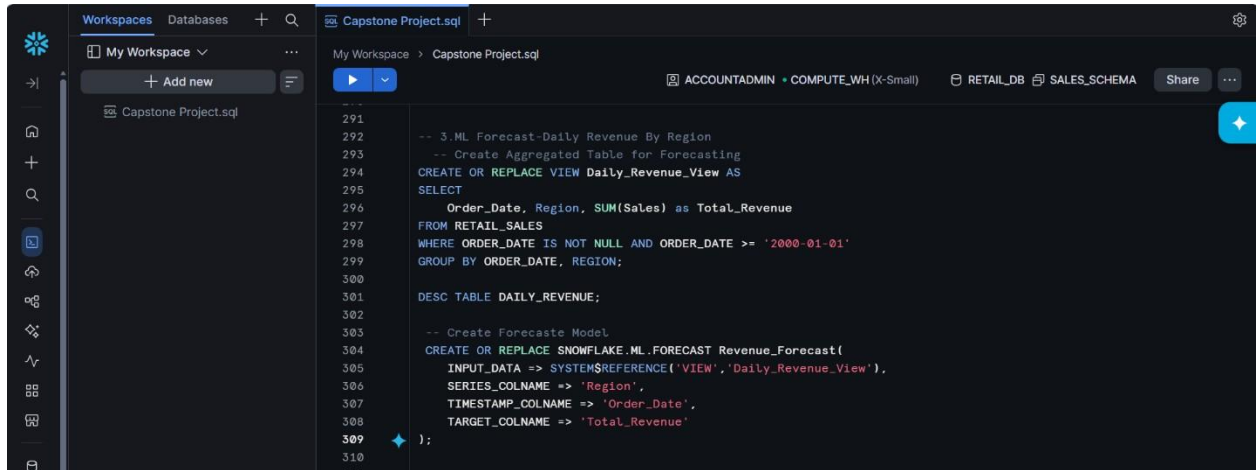
2. Cortex LLM - Summarize Insights

This leverages Snowflake Cortex LLM functions to automatically generate natural language narrative summaries of revenue trends. It dynamic feeds aggregated data directly into the Large Language Model to assist business users over the last 6 months



3. ML Forecast-Daily Revenue by Region

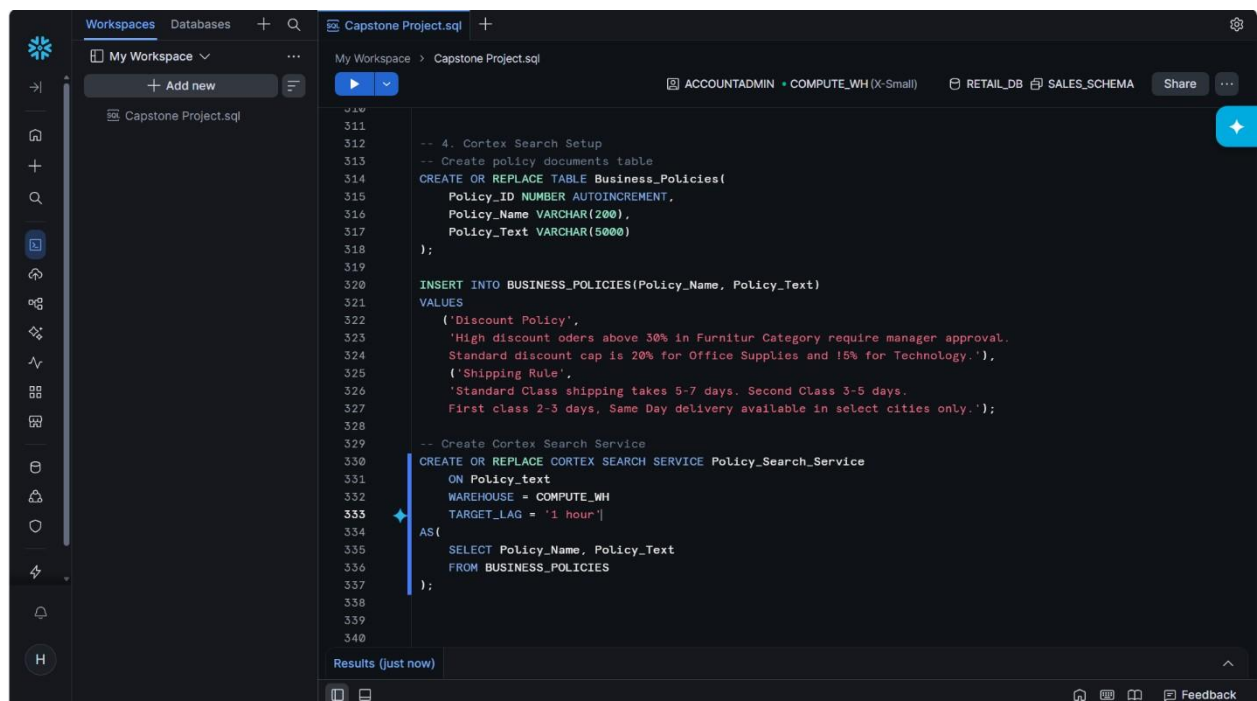
This creates an aggregated historical baseline view of daily revenue partitioned out by geographic region. It then trains a Snowflake Cortex Machine Learning model to forecast future expected daily revenue.



```
291
292 -- 3. ML Forecast-Daily Revenue By Region
293 -- Create Aggregated Table for Forecasting
294 CREATE OR REPLACE VIEW Daily_Revenue_View AS
295 SELECT
296     Order_Date, Region, SUM(Sales) as Total_Revenue
297 FROM RETAIL_SALES
298 WHERE ORDER_DATE IS NOT NULL AND ORDER_DATE >= '2000-01-01'
299 GROUP BY ORDER_DATE, REGION;
300
301 DESC TABLE DAILY_REVENUE;
302
303 -- Create Forecast Model
304 CREATE OR REPLACE SNOWFLAKE.ML.FORECAST Revenue_Forecast(
305     INPUT_DATA => SYSTEM$REFERENCE('VIEW', 'Daily_Revenue_View'),
306     SERIES_COLNAME => 'Region',
307     TIMESTAMP_COLNAME => 'Order_Date',
308     TARGET_COLNAME => 'Total_Revenue'
309 );
310
```

4. Cortex Search Setup

This creates text document describing “Discount policy” or “Shipping rules” like What is the policy for high discount orders in furniture category.



```
311
312 -- 4. Cortex Search Setup
313 -- Create policy documents table
314 CREATE OR REPLACE TABLE Business_Policies(
315     Policy_ID NUMBER AUTOINCREMENT,
316     Policy_Name VARCHAR(200),
317     Policy_Text VARCHAR(5000)
318 );
319
320 INSERT INTO BUSINESS_POLICIES(Policy_Name, Policy_Text)
321 VALUES
322     ('Discount Policy',
323      'High discount orders above 30% in Furniture Category require manager approval.
324       Standard discount cap is 20% for Office Supplies and 15% for Technology.'),
325     ('Shipping Rule',
326      'Standard Class shipping takes 5-7 days. Second Class 3-5 days.
327       First class 2-3 days, Same Day delivery available in select cities only.');
```

```
329 -- Create Cortex Search Service
330 CREATE OR REPLACE CORTEX SEARCH SERVICE Policy_Search_Service
331 ON Policy_text
332 WAREHOUSE = COMPUTE_WH
333 TARGET_LAG = '1 hour'
334 AS(
335     SELECT Policy_Name, Policy_Text
336     FROM BUSINESS_POLICIES
337 );
338
339
340
```

Results (just now)